Solving Data Skew: AQE & Broadcast Joins

Study notes for the video + notebook `spark/1_data_skew/3_solving_data_skew_aqe_broadcast.ipynb` (afaqueahmad7117/spark-experiments). Plans and Spark UI numbers come from a local run: Docker, Spark 4, `local[4]`, 39,790,092 transactions × 5,000 customers.

Join (same `count()`)	Time	Join tasks	Biggest task	Plan keyword
Sort merge join, AQE off	7.1 s	200	17,684,814 rows · 4.1 s	`SortMergeJoin`
Sort merge join, AQE default	6.8 s	4	17.7 M rows · 4.4 s	`AQEShuffleRead coalesced`
Sort merge join, AQE + 1 MB skew threshold	5.4–7.2 s	14	7.1 M rows · 1.6 s	`SortMergeJoin(skew=true)`
Broadcast hash join	**1.9 s**	12 (no shuffle)	3.7 M rows · 0.6 s	`BroadcastHashJoin`

- **Handwritten boxes** explain the cell above; red circles ①② in plans match the numbered bullets.
- The `.explain()` cells were added for these notes. Page 10 (landscape) = sort merge vs broadcast visuals. Pages 11–13 = **Spark UI walkthrough**: real screenshots of the job timeline, DAGs, stage metrics and task timelines, mapped to cells and plan lines. Typed pages at the end = Spark UI map, video recap, interview Q&A.

Imports & Configuration

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'

import warnings
warnings.filterwarnings('ignore')
warnings.simplefilter('ignore')
import time
```

```
from pyspark.sql.types import IntegerType
import pyspark.sql.functions as F
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.master("local[4]").getOrCreate()
sc = spark.sparkContext
sc.setLogLevel("ERROR")
```

local[4] = one machine, 4 cores

- `master("local[4]")` ⟶ the driver runs the tasks itself ⟶ Spark UI Executors tab shows only **driver** (4 cores)
- `getOrCreate()` ⟶ starts Spark + the UI at **localhost:4040**; no data read yet

💡 *No executors listed ≠ broken. On a real cluster you'd see executor 1…N.*

```
# spark.conf.set("spark.sql.shuffle.partitions", "3")
spark.conf.set("spark.sql.adaptive.enabled", "false")
```

Step 1: see the skew raw

- `adaptive.enabled = false` ⟶ turn AQE off so the join runs the plain way first

Join Skews

```
transactions_file = "../../data/data_skew/transactions.parquet"
customer_file = "../../data/data_skew/customers.parquet"

df_transactions = spark.read.parquet(transactions_file)
df_customers = spark.read.parquet(customer_file)
```

Reading is lazy

- `spark.read.parquet(...)` → Spark UI jobs 0 & 1: 1 tiny task each → reads only the **schema** (Parquet footer), not rows
- `df_transactions` → is a **recipe**, not data. Every action (show / count) re-reads the files

💡 Want to keep data between actions? **df.cache()** + one count() to fill it → shows in the Storage tab.

```
df_transactions.printSchema()
df_transactions.show(5, False)
```

```
root
 |-- cust_id: string (nullable = true)
 |-- start_date: string (nullable = true)
 |-- end_date: string (nullable = true)
 |-- txn_id: string (nullable = true)
 |-- date: string (nullable = true)
 |-- year: string (nullable = true)
 |-- month: string (nullable = true)
 |-- day: string (nullable = true)
 |-- expense_type: string (nullable = true)
 |-- amt: string (nullable = true)
 |-- city: string (nullable = true)

+----------+----------+----------+---------------+----------+----+-----+---+-------------+------+-----------+
|cust_id   |start_date|end_date  |txn_id         |date      |year|month|day|expense_type |amt   |city       |
+----------+----------+----------+---------------+----------+----+-----+---+-------------+------+-----------+
|C0YDPQWPBJ|2010-07-01|2018-12-01|TZ5SMKZY9S03OQJ|2018-10-07|2018|10   |7  |Entertainment|10.42 |boston     |
|C0YDPQWPBJ|2010-07-01|2018-12-01|TYIAPPNU066CJ5R|2016-03-27|2016|3    |27 |Motor/Travel |44.34 |portland   |
|C0YDPQWPBJ|2010-07-01|2018-12-01|TETSXIK4BLXHJ6W|2011-04-11|2011|4    |11 |Entertainment|3.18  |chicago    |
|C0YDPQWPBJ|2010-07-01|2018-12-01|TQKL1QFJY3EM8LO|2018-02-22|2018|2    |22 |Groceries    |268.97|los_angeles|
|C0YDPQWPBJ|2010-07-01|2018-12-01|TYL6DFP09PPXMVB|2010-10-16|2010|10   |16 |Entertainment|2.66  |chicago    |
+----------+----------+----------+---------------+----------+----+-----+---+-------------+------+-----------+
only showing top 5 rows
```

```
df_customers.printSchema()
df_customers.show(5, False)
```

```
root
 |-- cust_id: string (nullable = true)
 |-- name: string (nullable = true)
 |-- age: string (nullable = true)
 |-- gender: string (nullable = true)
 |-- birthday: string (nullable = true)
 |-- zip: string (nullable = true)
 |-- city: string (nullable = true)
```

```
+----------+-------------+---+------+----------+-----+-----------+
|cust_id   |name         |age|gender|birthday  |zip  |city       |
+----------+-------------+---+------+----------+-----+-----------+
|C007YEYTX9|Aaron Abbott |34 |Female|7/13/1991 |97823|boston     |
|C00B971T1J|Aaron Austin |37 |Female|12/16/2004|30332|chicago    |
|C00WRSJF1Q|Aaron Barnes |29 |Female|3/11/1977 |23451|denver     |
|C01AZWQMF3|Aaron Barrett|31 |Male  |7/9/1998  |46613|los_angeles|
|C01BKUFRHA|Aaron Becker |54 |Male  |11/24/1979|40284|san_diego  |
+----------+-------------+---+------+----------+-----+-----------+
only showing top 5 rows
```

```python
(
    df_transactions
    .groupBy("cust_id")
    .agg(F.countDistinct("txn_id").alias("ct"))
    .orderBy(F.desc("ct"))
    .show(5, False)
)
```

```
+----------+--------+
|cust_id   |ct      |
+----------+--------+
|C0YDPQWPBJ|17539732 ②|
|CBW3FMEAU7|7999    |
|C3KUDEN3KO|7999    |
|C89FCEGPJP|7999    |
|CHNFNR89ZV|7998    |
+----------+--------+
only showing top 5 rows
```

match the circles in the output above

Find the skewed key

- ① `C0YDPQWPBJ` → one customer…
- ② `17539732` → …has 17.5 M of the 39.8 M rows (≈ 44 %). Next biggest = 7,999
- `groupBy + countDistinct` → Spark UI job 4 → stages 4, 5, 6 (scan 12 tasks → 200 → 200), the slowest cell: 38.6 s

⚠ **Gotcha:** hash(cust_id) mod 200 → **all** 17.5 M rows of this key land in **one** shuffle partition = one task.

```python
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", -1)
```

Force a sort merge join

- `autoBroadcastJoinThreshold = -1` ⟶ -1 = never broadcast, so Spark must shuffle both sides

```
df_txn_details = (
    df_transactions.join(
        df_customers,
        on="cust_id",
        how="inner"
    )
)
```

```
start_time = time.time()
df_txn_details.count()
print(f"time taken: {time.time() - start_time}")
```

```
39790092
time taken: 7.105409860610962
```

Sort merge join, AQE off: 7.1 s

- `df_txn_details.count()` ⟶ Spark UI job 5 ⟶ stages 7 (scan txns, 12 tasks) · 8 (scan customers, 1) · 9 (join, **200** tasks) · 10 (final count, 1)
- `stage 9 Summary Metrics` ⟶ median task **110,390 rows · 0.02 s** vs max **17,684,814 rows · 4.1 s** ⟶ that's the skew

💡 Skew never shows in the plan. It shows in **Stages ⟶ Summary Metrics: Max ≫ Median.**

```
# AQE off, broadcast off
df_txn_details.groupBy().count().explain()
```

```
== Physical Plan ==
*(6) HashAggregate(keys=[], functions=[count(1)])
+- Exchange SinglePartition ⑤, ENSURE_REQUIREMENTS, [plan_id=80]
   +- *(5) HashAggregate(keys=[], functions=[partial_count(1)])
      +- *(5) Project
         +- *(5) SortMergeJoin [cust_id#0], [cust_id#11], Inner ④
            :- *(2) Sort [cust_id#0 ASC NULLS FIRST] ③, false, 0
            :  +- Exchange hashpartitioning(cust_id#0, 200) ②, ENSURE_REQUIREMENTS, [plan_id=62]
            :     +- *(1) Filter isnotnull(cust_id#0)
            :        +- *(1) ColumnarToRow
            :           +- FileScan parquet [cust_id#0] ① …
            +- *(4) Sort [cust_id#11 ASC NULLS FIRST], false, 0
               +- Exchange hashpartitioning(cust_id#11, 200), ENSURE_REQUIREMENTS, [plan_id=71]
                  +- *(3) Filter isnotnull(cust_id#11)
                     +- *(3) ColumnarToRow
                        +- FileScan parquet [cust_id#11] …
```

match the circles in the output above

Read the plan bottom → top

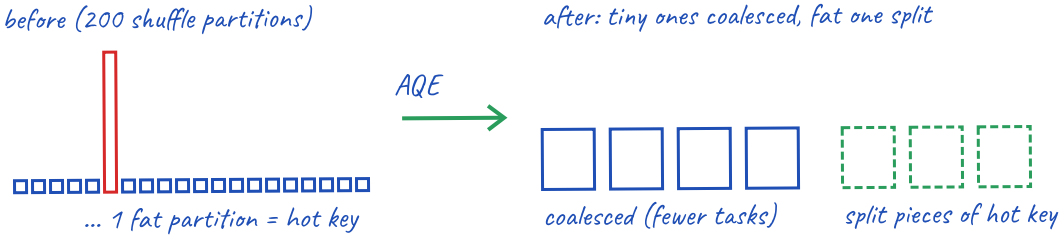
- ① read only **cust_id**: count() needs no other column (column pruning)
- ② **shuffle**: hash(cust_id) mod 200 → 200 partitions. Every Exchange = a stage boundary in the UI
- ③ **sort** each partition by cust_id
- ④ **merge**: walk both sorted sides like a zipper, emit matches
- ⑤ gather the 200 partial counts into 1 (stage 10)

💡 *(n) = whole-stage codegen: operators with the same number are fused into one compiled function.

Using AQE

✍️ Definition: AQE = Adaptive Query Execution (Spark 3.0+)

- after each shuffle stage, Spark looks at **runtime stats** (bytes per partition) and re-plans the rest
- ① **coalesce** tiny shuffle partitions (200 → 4 in our run)
- ② **switch** sort merge → broadcast if a side turns out small
- ③ **split skewed partitions** (skew join) and copy the matching rows of the other side to each piece
- skewed ⟺ size > **5** × **median** AND > **256 MB** (both!)



💡 AQE decides at runtime, so explain() before an action shows isFinalPlan=false.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")   # notebook typo: skewedJoin
# added for these notes: count() shuffles only cust_id (~3.5 MB), below the 256 MB default
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes", "1MB")
spark.conf.set("spark.sql.adaptive.advisoryPartitionSizeInBytes", "512KB")
```

The settings (fixed + tuned)

- `skewJoin.enabled` → notebook had a typo `skewedJoin` → silently ignored (works anyway: default = true)
- `skewedPartitionThresholdInBytes 1MB` → count() shuffles only cust_id → the fat partition is just **3.5 MB** < 256 MB default → never flagged
- `advisoryPartitionSizeInBytes 512KB` → target size of each split piece

⚠ **Gotcha:** The 6.77 s output below came from the **default** settings: AQE only coalesced, it did not split. Same as in the video (4 tasks, max 7 s).

```
df_txn_details = (
    df_transactions.join(
        df_customers,
        on="cust_id",
        how="inner"
    )
)
```

```
start_time = time.time()
df_txn_details.count()
print(f"time taken: {time.time() - start_time}")
```

```
39790092
time taken: 6.7658851146698
```

```
# AQE on, default skew settings (after the count ran)
df_txn_details.groupBy().count().explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=true ①
+- == Final Plan ==
   ResultQueryStage 3
   +- *(6) HashAggregate(keys=[], functions=[count(1)])
      +- ShuffleQueryStage 2
         +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=298]
            +- *(5) HashAggregate(keys=[], functions=[partial_count(1)])
               +- *(5) Project
                  +- *(5) SortMergeJoin [cust_id#0], [cust_id#11], Inner ③
                     :- *(3) Sort [cust_id#0 ASC NULLS FIRST], false, 0
                     :  +- AQEShuffleRead coalesced ②
                     :     +- ShuffleQueryStage 0
                     :        +- Exchange hashpartitioning(cust_id#0, 200), ENSURE_REQUIREMENTS, [plan_id=190]
                     :           +- *(1) Filter isnotnull(cust_id#0)
                     :              +- *(1) ColumnarToRow
                     :                 +- FileScan parquet [cust_id#0] …
                     +- *(4) Sort [cust_id#11 ASC NULLS FIRST], false, 0
                        +- AQEShuffleRead coalesced
                           +- ShuffleQueryStage 1
                              +- Exchange hashpartitioning(cust_id#11, 200), ENSURE_REQUIREMENTS, [plan_id=207]
                                 +- *(2) Filter isnotnull(cust_id#11)
                                    +- *(2) ColumnarToRow
                                       +- FileScan parquet [cust_id#11] …
+- == Initial Plan ==  (trimmed: same tree without AQE nodes)
```

match the circles in the output above

AQE with default settings

- ① final plan, built from runtime stats
- ② 200 partitions read as 4 (coalesced only)
- ③ no (skew=true) ⟶ skew not handled
- **stage 15** ⟶ 4 tasks · median 9.7 M rows / 2.3 s · max 17.7 M rows / 4.4 s · total 6.8 s

```
# AQE on + skewedPartitionThresholdInBytes=1MB, advisory=512KB
df_txn_details.groupBy().count().explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
   ResultQueryStage 3
   +- *(6) HashAggregate(keys=[], functions=[count(1)])
      +- ShuffleQueryStage 2
         +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=536]
            +- *(5) HashAggregate(keys=[], functions=[partial_count(1)])
               +- *(5) Project
                  +- *(5) SortMergeJoin(skew=true) ① [cust_id#0], [cust_id#11], Inner
                     :- *(3) Sort [cust_id#0 ASC NULLS FIRST], false, 0
                     :  +- AQEShuffleRead coalesced and skewed ②
                     :     +- ShuffleQueryStage 0 ③
                     :        +- Exchange hashpartitioning(cust_id#0, 200), ENSURE_REQUIREMENTS, [plan_id=398]
                     :           +- *(1) Filter isnotnull(cust_id#0)
                     :              +- *(1) ColumnarToRow
                     :                 +- FileScan parquet [cust_id#0] …
                     +- *(4) Sort [cust_id#11 ASC NULLS FIRST], false, 0
                        +- AQEShuffleRead coalesced
                           +- ShuffleQueryStage 1
                              +- Exchange hashpartitioning(cust_id#11, 200), ENSURE_REQUIREMENTS, [plan_id=415]
                                 +- *(2) Filter isnotnull(cust_id#11)
                                    +- *(2) ColumnarToRow
                                       +- FileScan parquet [cust_id#11] …
+- == Initial Plan ==  (trimmed: same tree without AQE nodes)
```

match the circles in the output above

AQE with the 1 MB threshold: skew split

- ① AQE marked the join as skewed
- ② fat partition **split** into ~512 KB pieces; 'coalesced' = small ones merged
- ③ query stage = AQE runs each shuffle as its own job ⟶ later jobs list them again as **skipped** stages
- **join stage** ⟶ 14 tasks · max 7.1 M rows / 1.6 s (was 17.7 M / 4.4 s) · total 5.4–7.2 s

💡 Only 4 cores, so 14 tasks still queue: small wall-time win here, big win on a real cluster.

Using Broadcast Joins

✍️ Definition: Broadcast hash join

- copy the **small** table (customers, 5,000 rows) to **every** executor
- build a hash table from it; stream the big table through it
- **no shuffle** of the big side → it can stay evenly split → **immune to skew**
- auto when a side < autoBroadcastJoinThreshold (10 MB) · or force: **F.broadcast(df)**

💡 Risk: the small side must fit in memory on the driver and every executor. Hundreds of MB → OOM.

```
# 10MB = 10485760 Bytes
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10485760)
```

Turn broadcast back on

- `10485760` → = 10 MB. Any join side smaller than this is broadcast automatically

```
df_txn_details = (
    df_transactions.join(
        F.broadcast(df_customers),
        on="cust_id",
        how="inner"
    )
)
```

```
start_time = time.time()
df_txn_details.count()
print(f"time taken: {time.time() - start_time}")
```

```
39790092
time taken: 1.8948593139648438
```

```
# broadcast join
df_txn_details.groupBy().count().explain()
```

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=true
+- == Final Plan ==
   ResultQueryStage 2
   +- *(3) HashAggregate(keys=[], functions=[count(1)])
      +- ShuffleQueryStage 1
         +- Exchange SinglePartition, ENSURE_REQUIREMENTS, [plan_id=681]
            +- *(2) HashAggregate(keys=[], functions=[partial_count(1)])
               +- *(2) Project
                  +- *(2) BroadcastHashJoin [cust_id#0], [cust_id#11], Inner, BuildRight ①, false, false
                     :- *(2) Filter isnotnull(cust_id#0)
                     :  +- *(2) ColumnarToRow
                     :     +- FileScan parquet [cust_id#0] ③ …
                     +- BroadcastQueryStage 0
                        +- BroadcastExchange HashedRelationBroadcastMode ②(List(input[0, string, false]),false),
[plan_id=625]
                           +- *(1) Filter isnotnull(cust_id#11)
                              +- *(1) ColumnarToRow
                                 +- FileScan parquet [cust_id#11] …
+- == Initial Plan ==  (trimmed: same tree without AQE nodes)
```

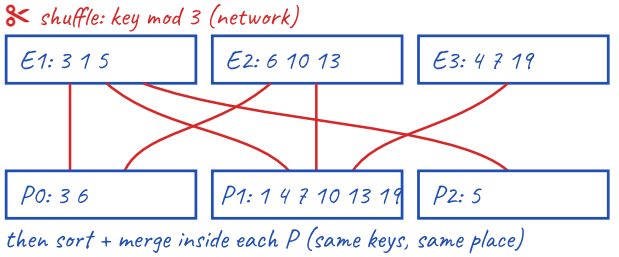
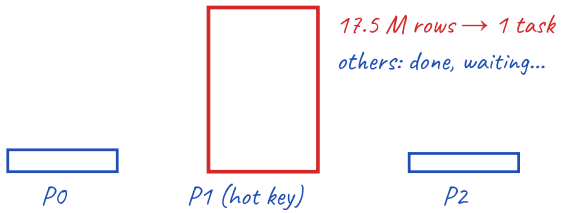
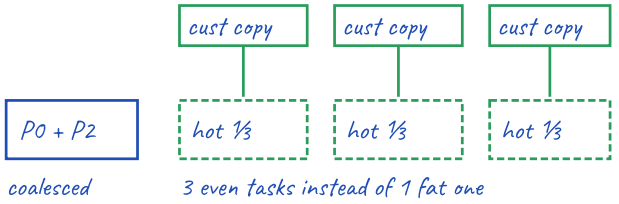
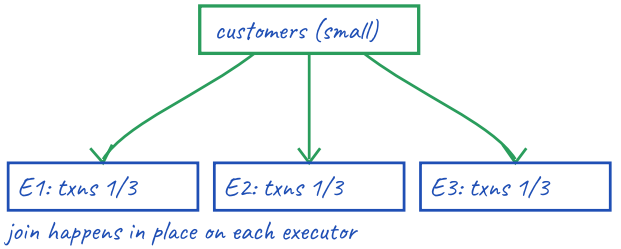
match the circles in the output above

Broadcast join: 1.9 s (vs 7.1 s)

- ① hash table built from the **right** side (customers)
- ② customers sent once to all tasks (replaces the shuffle Exchange)
- ③ transactions read in place: **no Exchange above it**
- **stage 21** ⟶ 12 tasks · median 3.4 M rows / 0.55 s · max 3.7 M / 0.60 s ⟶ even

💡 The hot key's 17.5 M rows stay spread over the 12 file pieces, so no task gets them all.

Sort merge join vs broadcast join: how the rows move

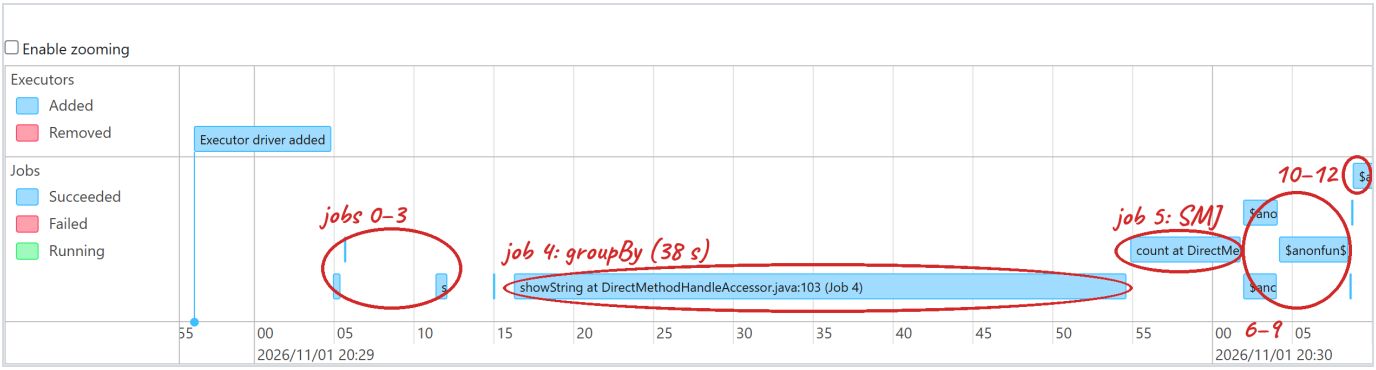
Technique	Rule	Worked example (3 executors, key mod 3)	Visual
Sort merge join `t.join(c, "cust_id")`	① **shuffle** both sides: partition = hash(key) mod n ② sort each partition ③ merge the two sorted lists	txns E1: 3 1 5 · E2: 6 10 13 · E3: 4 7 19 ⟶ P0 = 3 6 · P1 = 1 4 7 10 13 19 · P2 = 5 customers go to the same P ⟶ matches meet	
Sort merge + skew	every row of one key ⟶ **one** partition ⟶ one task. Other cores finish and sit idle	ours: COYDPQWPBJ = 17.5 M rows ⟶ 1 of 200 tasks reads 17,684,814 rows (4.1 s) vs median 0.02 s	
AQE skew join `adaptive.skewJoin.enabled`	skewed ⟺ > 5 × median **and** > threshold (256 MB) ⟶ **split** into ~advisory-size pieces, **copy** the matching rows of the other side to each piece; tiny partitions coalesced	fat partition was only 3.5 MB ⟶ not flagged by default. With threshold 1 MB / advisory 512 KB: 14 tasks, biggest 7.1 M rows (1.6 s)	
Broadcast hash join `t.join(F.broadcast(c), "cust_id")`	copy the **small** side to every executor, hash it, stream the big side through. **No shuffle** of the big side ⟶ keys need not be grouped ⟶ **immune to skew**	5,000 customers sent to all 12 tasks; transactions stay as read (or `repartition(3)`) ⟶ even: max 3.7 M vs median 3.4 M rows · 1.9 s	

💡 Same answer, different cost: sort merge moves **both** tables by key (and the hot key piles up); broadcast moves only the **small** one.

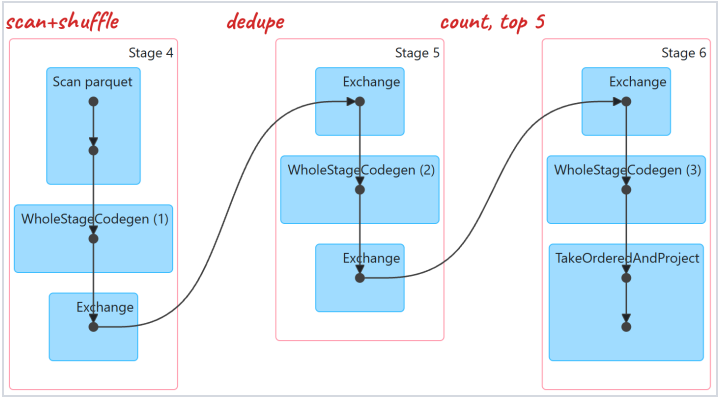
✍️ Spark UI walkthrough: cell → job → stage → task

- **Jobs tab → Event Timeline:** one bar per **job** (= one action: show / count). Circled below by cell.
- **Click a job → DAG Visualization:** red box = **stage**, blue box = operator group, **grey = skipped**. Every arrow between stages = an **Exchange** = shuffle.
- **WholeStageCodegen (n)** in the DAG = ***(n)** in the physical plan. That's how plan lines map to DAG boxes.
- **Stages tab → click a stage:** Summary Metrics (compare **Median** vs **Max**) + Event Timeline (one bar per **task**; 4 rows of slots = 4 cores).

💡 Read it top-down: timeline (which job) → DAG (which stages, grey = reused) → stage metrics (Max ≫ Median?) → task timeline (one long bar?).



Circle	Cell	Stages	What happened
jobs 0–3	read.parquet ×2, show ×2	0, 1, 2, 3	1 task each: schema, then one 4,096-row batch
job 4	groupBy … countDistinct	4, 5, 6	the long bar: 20:29:16 → 20:29:54 (38 s)
job 5	SMJ count, AQE off	7, 8, 9, 10	20:29:54 → 20:30:01 (7 s)
jobs 6–9	SMJ count, AQE on	11–19	AQE: one job per query stage. 6 + 7 start together
jobs 10–12	broadcast count	20–23	20:30:08 → 20:30:10 (2 s)

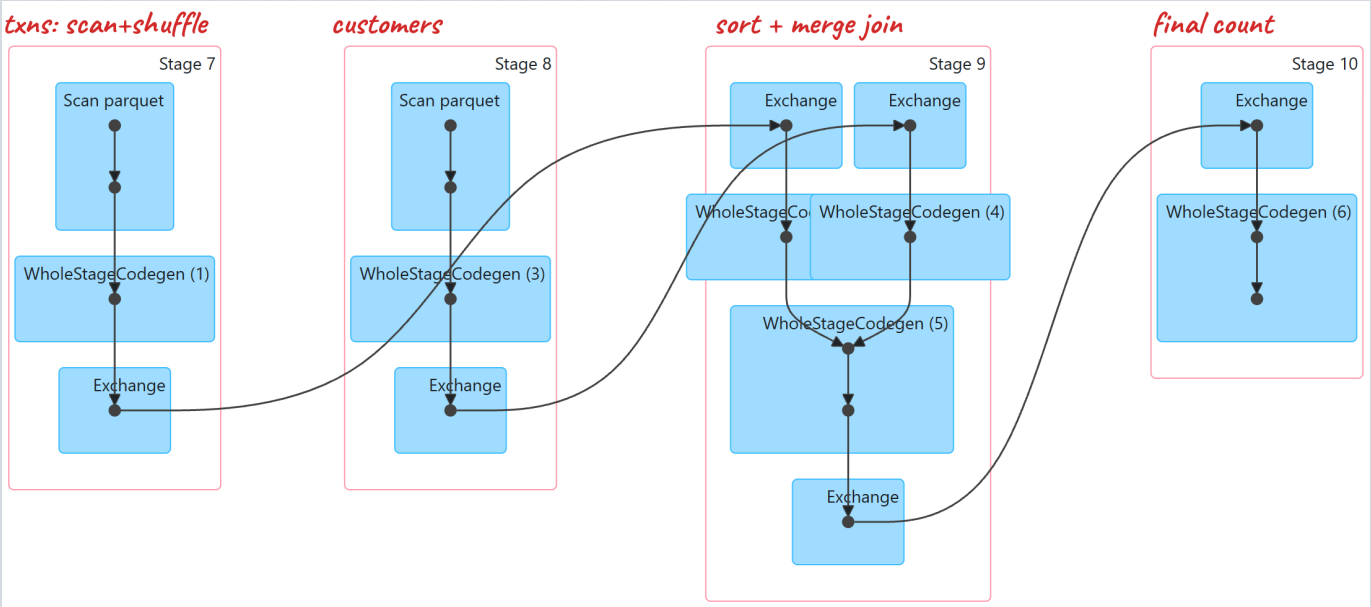


job 4 DAG (cell 10: groupBy)

- **Stage 4:** scan 12 file pieces, partial agg, **shuffle** by (cust_id, txn_id)
- **Stage 5:** 200 tasks remove duplicate txn_ids, shuffle by cust_id
- **Stage 6:** 200 tasks count + **TakeOrderedAndProject** = orderBy + show(5)

💡 2 Exchanges → 3 stages.

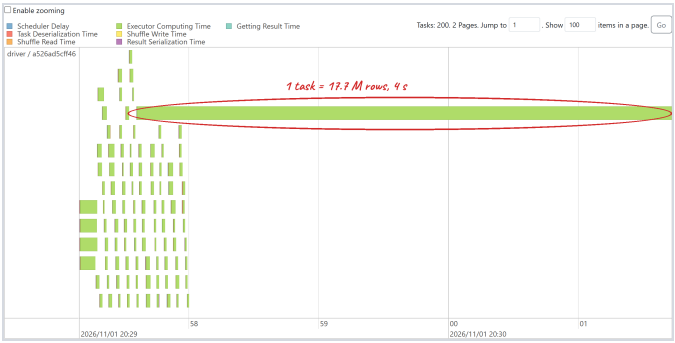
Sort merge join, AQE off: cell 13 → job 5



job 5 DAG: stages 7 → 9 ← 8, then 10

Metric	Median	Max
Duration	20.0 ms	4 s
GC Time	0.0 ms	84.0 ms
Spill (memory)	0.0 B	508 MiB
Spill (disk)	0.0 B	3.5 MiB
Shuffle Read Size / Records	23.4 KiB / 110390	3.5 MiB / 17684814
Shuffle Write Size / Records	59 B / 1	59 B / 1

stage 9 Summary Metrics (Metric | Median | Max)

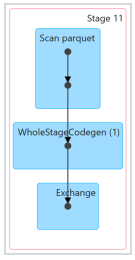


stage 9 task timeline (200 tasks)

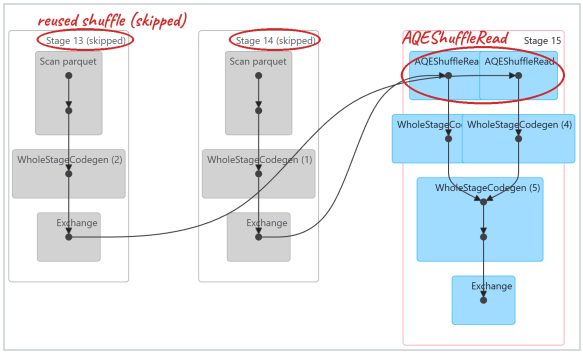
Stage	Plan lines (match *(n))	UI evidence
7	*(1) FileScan txns → Exchange hashpartitioning(200)	12 tasks write 39.8 M rows to shuffle
8	*(3) FileScan customers → Exchange	1 task, 5,000 rows
9	*(2) Sort · *(4) Sort · *(5) SortMergeJoin + partial count	200 tasks · median **110,390 rows / 20 ms** · max **17,684,814 rows / 4 s**
10	Exchange SinglePartition → *(6) count	1 task adds the 200 partial counts

⚠ **New from the UI:** the hot-key task **spilled 508 MiB** from memory (3.5 MiB on disk) because its sort buffer overflowed. The median task spilled 0.

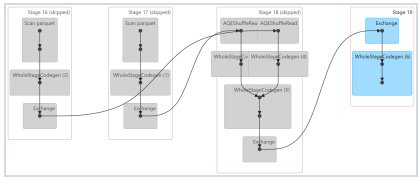
AQE on (default settings): cell 17 → jobs 6–9



job 6



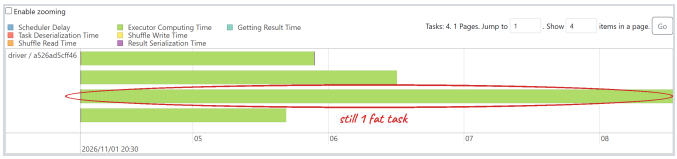
job 8: stages 13, 14 skipped → 15



job 9: 16–18 skipped → 19

Metric	Median	Max
Duration	2 s	4 s
GC Time	0.2 s	0.2 s
Spill (memory)	312 MiB	616 MiB
Spill (disk)	2.1 MiB	4.2 MiB
Shuffle Read Size / Records	2 MiB / 9676283	3.5 MiB / 17684814
Shuffle Write Size / Records	59 B / 1	59 B / 1

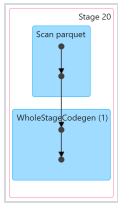
stage 15 Summary Metrics



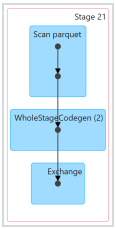
stage 15 task timeline (4 tasks)

- **job 6 = stage 11** (txns shuffle) and **job 7 = stage 12** (customers shuffle) start at the same second: AQE submits each query stage as its own job, then reads their sizes.
- **job 8:** stages 13 and 14 are the same shuffles again → **skipped** (files reused). Stage 15 reads them through `AQEShuffleRead` → 4 coalesced partitions.
- **job 9:** everything before is skipped; stage 19 = final count.
- **Stage 15:** median 9.7 M rows / 2 s vs max **17.7 M / 4 s**. Not split. All 4 tasks spill 208–616 MiB, because merging partitions made every task big.

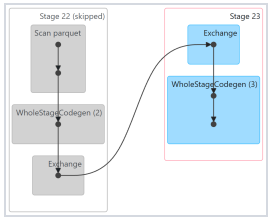
Broadcast join: cell 21 → jobs 10–12



job 10



job 11

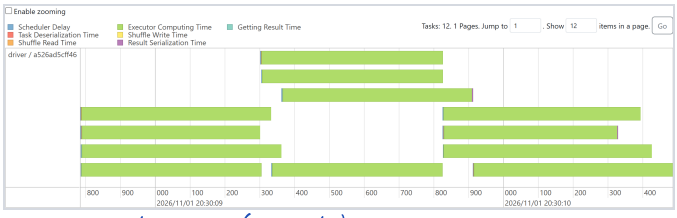


job 12

- **job 10 = stage 20:** read customers, build the broadcast (1 task)
- **job 11 = stage 21:** scan txns → `WholeStageCodegen (2)` = plan `*(2) BroadcastHashJoin` + partial count. Its Exchange carries 12 tiny rows, not data
- **job 12:** 22 skipped, 23 = final count
- **Stage 21:** median **3.4 M** vs max **3.66 M** rows, 0.5–0.6 s, **no spill** → even. No join-key shuffle for skew to pile up in.

Metric	Median	Max
Duration	0.5 s	0.6 s
GC Time	17.0 ms	23.0 ms
Input Size / Records	85.3 KiB / 3408595	110.2 KiB / 3663839
Shuffle Write Size / Records	59 B / 1	59 B / 1

stage 21 Summary Metrics



stage 21 task timeline (12 tasks)

Typed notes: Spark UI, video recap, interview prep

Numbers are from the Spark UI of the notebook run (localhost:4040, 4 cores). Same dataset everywhere: 39.8 M transactions, 5,000 customers, hot key C0YDPQWPBJ.

1. Spark UI map: which cell made which jobs and stages

Cell	Jobs	Stages	What the stages do
`spark.read.parquet` ×2	0, 1	0, 1	1 task each: read the schema (footer) only
`.show(5)` ×2	2, 3	2, 3	1 task: one 4,096-row batch, then stop
`groupBy … countDistinct … show`	4	4, 5, 6	scan + shuffle (12 tasks) → dedupe (200) → count + top 5 (200); 38.6 s
SMJ `count()`, AQE off	5	7, 8, 9, 10	shuffle txns (12) · shuffle customers (1) · **join (200)** · final count (1)
SMJ `count()`, AQE on	6, 7, 8, 9	11–19	11, 12 shuffles · 15 join (**4** tasks) · 19 final count · 13, 14, 16, 17, 18 **skipped**
Broadcast `count()`	10, 11, 12	20–23	20 read customers to broadcast · 21 join in place (12) · 23 final count · 22 skipped

- **Job** = one action. **Stage** = the work between two shuffles (each `Exchange` in the plan). **Task** = one partition of a stage.
- **Executors tab shows only "driver"**: in `local[4]` the driver is the executor. **Storage tab empty**: nothing was `.cache()`d.
- **24 stages = 18 run + 6 skipped.** AQE runs each shuffle as its own job, and later jobs list those stages again. Their shuffle files already exist, so they're **skipped** (reused, not lost work).
- Names like `showString at …` and `$anonfun$withThreadLocalCaptured` (AQE jobs) are internal. Set your own with `sc.setJobDescription("SMJ - AQE off")` before the action.

> One-liner: every Exchange in the plan is a stage boundary, and every action is a job.

★ 2. Where skew shows up: Stages → Summary Metrics

Skew = Max task ≫ Median task in a shuffle stage

The physical plan looks the same with or without skew. Only the task metrics show it.

Join stage	Tasks	Median rows · time	Max rows · time	Verdict
9: SMJ, AQE off	200	110,390 · 0.02 s	**17,684,814 · 4.1 s**	skewed
15: SMJ, AQE default	4	9.7 M · 2.3 s	**17.7 M · 4.4 s**	coalesced, still skewed
SMJ, AQE + 1 MB threshold	14	1.7 M · 0.56 s	7.1 M · 1.6 s	**split**
21: broadcast	12	3.4 M · 0.55 s	3.7 M · 0.60 s	**even**

Also check: the **Event Timeline** (one long bar, many tiny ones), and the Tasks table sorted by Duration (the top task holds the hot key). **Spill > 0** on that task means it ran out of memory.

> Gotcha: `skewedPartitionThresholdInBytes` is measured in **shuffle bytes**. `count()` only shuffles `cust_id`, so 17.5 M rows were just 3.5 MB, far under the 256 MB default. A query that selects more columns makes the partition bigger, so AQE's default would catch it more easily.

3. Video recap: key points (and what our run showed)

- **Two fixes for skew:** AQE (Spark 3.0+) and broadcast joins. Salting is the third, in a separate video.
- **AQE uses runtime statistics** (bytes read, number of partitions) to pick a better plan. It does three things: coalesce shuffle partitions (e.g. 200 → 15 if there are only 15 distinct keys), switch sort merge → broadcast, and split skewed partitions.
- **Docs:** needs `spark.sql.adaptive.enabled` **and** `spark.sql.adaptive.skewJoin.enabled`. Skewed = > `skewedPartitionFactor` (5) × median **and** > `skewedPartitionThresholdInBytes` (256 MB). `forceOptimizeSkewedJoin` (3.3+) applies the fix even if it adds a shuffle.
- **Hash partitioning:** partition = `hash(key) mod shuffle_partitions`. The hash makes any type (string, array, several keys) into an integer, and the mod puts it in a bucket.
- **Sort merge join** = shuffle → sort → merge. The shuffle is the costly step (network transfer).
- **Broadcast join** sends one copy of the small table to every executor. The big table can be split however you like (e.g. `repartition(3)`) instead of by join key, so there's no skew.
- **Video's times:** AQE 11.8 → 10.9 s (small gain), broadcast 3.2 s. **Ours:** 7.1 → 6.8 s, broadcast 1.9 s. The same pattern: in both, AQE only coalesced (4 tasks), and the max task stayed long.

> One-liner: AQE patches the shuffle after it happens; broadcast avoids the shuffle altogether.

4. Check your understanding (your own explanations)

You said		Better way to say it
"Every cell is a new script that deletes everything"	✗	The session, variables, settings, shuffle files and caches stay. Only the computed rows aren't kept unless you `.cache()`.
"show() is an action; groupBy / join are recipes"	✓	Transformations return a DataFrame (lazy). Actions return something else or write data, and start a job.
"Partition size is 0–200 MB"	✗	Aim for ~100–200 MB. Read splits default to 128 MB; AQE targets 64 MB; 200 is the shuffle partition **count**.
"Partitions = 2–4 × cores; that forms waves"	✓	waves = tasks ÷ cores. One huge task still stalls every wave.
"DuckDB is Spark at a smaller scale"	✗	DuckDB is Spark's optimizer on one machine: no cluster, no network shuffle, often faster on small data.
"Parquet footer maps row group → byte address"	✓	One footer at the end lists every row group's offsets and min/max. 1 row group = 1 thread for that file.

5. Interview questions

Q1. Your join has one task running 100× longer than the rest. What's going on and how do you fix it?

> *That's data skew: one join key has far more rows, and hash partitioning sends them all to one task. I'd confirm it in the Stages tab (max vs median task, shuffle read records). If the other side is small, I'd broadcast it, which removes the shuffle. Otherwise I'd enable AQE skew join and check that the partition is over the skew threshold, or salt the hot key.*

Q2. Why is a broadcast join immune to skew?

> *Skew comes from partitioning by the join key. In a broadcast join the big side is never shuffled by key; every task already has the full small table, so the hot key's rows stay spread across the original partitions.*

Q3. You turned on AQE but the skewed task didn't get split. Why?

> *AQE flags a partition as skewed only if it's larger than 5× the median and larger than skewedPartitionThresholdInBytes, 256 MB by default. If the shuffled data is small, for example because only the key column is shuffled, it never qualifies. Lowering the threshold, or the advisory partition size, lets it split.*

Q4. When should you not broadcast?

> *When the "small" side is large: hundreds of MB or more. It's collected on the driver and copied to every executor, so it can cause out-of-memory errors. Also for full outer joins, which can't be broadcast.*

> One-liner: find skew with max vs median task metrics; fix it by broadcasting, AQE skew join, or salting.

6. Not covered yet → study next

- **Salting**: add a random suffix to the hot key, explode the small side to match, join, then drop the suffix.
- **Shuffled hash join** (`spark.sql.join.preferSortMergeJoin=false`) and join hints: `/*+ BROADCAST */` , `MERGE` , `SHUFFLE_HASH` .
- `spark.sql.adaptive.forceOptimizeSkewedJoin` (3.3+), and AQE `coalescePartitions.minPartitionSize` .
- Caching (`.cache()` , the Storage tab) and how `InMemoryTableScan` replaces `FileScan` in plans.
- Databricks: the Query Profile instead of the Spark UI on serverless, where AQE is always on.